Работа с БД. ADO.NET

1. Что такое база данных

Практически любое современное приложение работает с данными, которые необходимо хранить между сессиями, обновлять и искать. Простейший способ — сохранять данные в файлы. Однако по мере роста приложения этот подход быстро перестаёт работать: файлы не поддерживают одновременный доступ нескольких пользователей, не умеют эффективно искать данные по условию, не защищают от частичной записи при сбое.

Для решения этих задач используются **базы данных** — специализированные системы хранения, которые обеспечивают надёжное, структурированное и эффективное управление информацией.

База данных и СУБД

Важно сразу разделить два понятия, которые часто путают.

База данных (БД) — это сами данные и их структура: таблицы, связи, ограничения.

Система управления базами данных (СУБД) — это программное обеспечение, которое хранит базу данных, обрабатывает запросы к ней, обеспечивает безопасность и целостность данных, управляет одновременным доступом нескольких клиентов.

Клиент–серверная модель

Большинство СУБД работают по **клиент–серверной модели**: сервер базы данных запущен отдельно, принимает подключения и обрабатывает запросы, а клиентское приложение подключается к нему по сети и отправляет SQL-запросы.

Однако существует и другой вариант — **встраиваемые СУБД**, которые работают прямо внутри приложения без отдельного сервера.

Обзор популярных СУБД

Сегодня существует множество СУБД, каждая со своими сильными сторонами и типичными сценариями применения.

PostgreSQL — мощная реляционная СУБД с открытым исходным кодом. Поддерживает сложные типы данных, оконные функции, полнотекстовый поиск, расширения. Широко используется в системах, где важны надёжность и функциональность. Хороший выбор по умолчанию для серверных приложений.

MySQL— одна из самых распространённых СУБД в веб-разработке. Традиционно используется в связке с PHP и является основой большинства CMS.

Microsoft SQL Server — коммерческая СУБД от Microsoft, плотно интегрированная в экосистему .NET. Имеет развитые инструменты администрирования (SQL Server Management Studio), хорошо подходит для корпоративных приложений на Windows.

SQLite заслуживает отдельного внимания, поскольку именно с ней мы будем работать в этой лекции.

SQLite — это встраиваемая реляционная СУБД, которая:

- не требует отдельного сервера — работает прямо внутри приложения;
- хранит всю базу данных в одном `.db`-файле;
- полностью поддерживает SQL;
- не требует установки и настройки.

SQLite является одной из самых широко используемых СУБД в мире — она встроена в Android, iOS, браузеры, многие десктопные приложения и является стандартным хранилищем для локальных данных.

2. Реляционные БД, проектирование и SQL

Вы уже знакомы с реляционными базами данных, нормализацией и SQL. Этот раздел — краткое напоминание ключевых концепций и справочная схема, к которой мы будем обращаться в остальных разделах лекции.

Реляционная модель представляет данные в виде **таблиц**: каждая строка — одна запись, каждый столбец — одно свойство. SQLite использует пять базовых типов: `INTEGER`, `REAL`, `TEXT`, `BLOB`, `NULL`.

Первичный ключ (`PRIMARY KEY`) однозначно идентифицирует строку. На практике используют суррогатный автоинкрементный идентификатор, а на поля с естественной уникальностью накладывают `UNIQUE`:

```
CREATE TABLE Customers (
    Id    INTEGER PRIMARY KEY AUTOINCREMENT, -- суррогатный ключ
    Email TEXT    NOT NULL UNIQUE            -- естественный ключ
);
```

Внешний ключ (`FOREIGN KEY`) ссылается на первичный ключ другой таблицы и обеспечивает ссылочную целостность. В SQLite поддержка внешних ключей по умолчанию отключена и включается через строку подключения или явно:

```
PRAGMA foreign_keys = ON;
```

Связи между таблицами выражаются через внешние ключи:

- **1:N** — самый распространённый тип. Внешний ключ размещается на стороне «многих»: один покупатель — много заказов.
- **N:M** — реализуется через промежуточную таблицу: один заказ — много товаров, один товар — во многих заказах.
- **1:1** — встречается редко, чаще данные объединяют в одну таблицу.

Ограничения целостности защищают данные на уровне схемы: `NOT NULL` — обязательное поле, `UNIQUE` — уникальность значения, `CHECK` — произвольное условие, `DEFAULT` — значение по умолчанию.

Нормализация устраняет дублирование данных и логические противоречия. На практике достаточно соблюдать три правила: каждая ячейка содержит одно значение (1НФ), данные зависят от всего первичного ключа, а не от его части (2НФ), неключевые столбцы зависят только от первичного ключа, а не друг от друга (3НФ).

SQL

SQL (Structured Query Language) — это универсальный язык программирования, используемый для управления, манипулирования и анализа данных, хранящихся в реляционных базах данных. Основные команды SQL чаще всего делят на две фундаментальные группы по функциональности: **DDL** и **DML** . Ниже — краткое напоминание ключевых команд.

DDL (Data Definition Language) — управление структурой:

```sql
-- Создать таблицу
CREATE TABLE IF NOT EXISTS Products (
    Id    INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT    NOT NULL,
    Price REAL    NOT NULL CHECK (Price > 0),
    Stock INTEGER NOT NULL DEFAULT 0
);

-- Добавить столбец
ALTER TABLE Customers ADD COLUMN Phone TEXT;

-- Удалить таблицу
DROP TABLE IF EXISTS Orders;
```

DML (Data Manipulation Language) — работа с данными:

```sql
-- Добавить запись
INSERT INTO Customers (Name, Email)
VALUES ('Иван Иванов', 'ivan@example.com');

-- Читать данные
SELECT Id, Name, Email FROM Customers
WHERE Email = 'ivan@example.com';

-- Сортировка, пагинация
SELECT Id, Title, Price FROM Products
ORDER BY Price DESC
LIMIT 10 OFFSET 0;

-- Агрегация
SELECT CustomerId, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerId;

-- JOIN — объединение таблиц
-- INNER JOIN: только совпадающие строки
SELECT Orders.Id, Customers.Name, Orders.Status
```

```sql
FROM Orders
INNER JOIN Customers ON Orders.CustomerId = Customers.Id;

-- LEFT JOIN: все строки левой таблицы, даже без совпадения справа
SELECT Orders.Id, Customers.Name, Orders.Status
FROM Orders
LEFT JOIN Customers ON Orders.CustomerId = Customers.Id;

-- Обновить запись
UPDATE Products
SET Price = 52000.0, Stock = 8
WHERE Id = 1;

-- Удалить запись
DELETE FROM Customers
WHERE Id = 3;
```

3. Введение в ADO.NET

ADO.NET (ActiveX Data Objects .NET) — это набор классов в стандартной библиотеке .NET, который предоставляет низкоуровневый доступ к реляционным базам данных. ADO.NET позволяет выполнять SQL-запросы, читать результаты, управлять транзакциями и соединениями — напрямую.

Провайдеры данных

ADO.NET спроектирован так, чтобы работать с любой СУБД через единый интерфейс. Конкретная реализация для каждой СУБД называется **провайдером данных** — это набор классов, реализующих абстракции ADO.NET для конкретной базы данных. Разные СУБД — разные провайдеры, но API остаётся одинаковым.

СУБД	NuGet-пакет
SQLite	`Microsoft.Data.Sqlite`
MS SQL Server	`Microsoft.Data.SqlClient`
PostgreSQL	`Npgsql`
MySQL	`MySqlConnector`

Установка пакета для SQLite:

```
dotnet add package Microsoft.Data.Sqlite
```

После установки все классы доступны через:

```
using Microsoft.Data.Sqlite;
```

Так же может понадобится установить пакет:

```
dotnet add package SQLitePCLRaw.bundle_e_sqlite3
```

Ключевое преимущество архитектуры с провайдерами: если в будущем нужно сменить СУБД, большая часть кода останется неизменной — поменяются только строка подключения и типы классов соединения.

Строка подключения

Прежде чем выполнить любой SQL-запрос, приложение должно установить соединение с базой данных. Для этого используется **строка подключения (connection string)** — набор пар `ключ=значение`, разделённых точкой с запятой, который содержит всю необходимую информацию: адрес сервера, имя базы, учётные данные.

Конкретный набор параметров зависит от СУБД:

```
# MS SQL Server
Server=localhost;Database=ShopDb;User Id=myuser;Password=mypassword;

# PostgreSQL
Host=localhost;Port=5432;Database=ShopDb;Username=myuser;Password=mypassword
;

# MySQL
Server=localhost;Database=ShopDb;User=myuser;Password=mypassword;
```

SQLite устроена проще — нет сервера, нет пользователей, нет сетевого адреса. Вся база хранится в одном файле, поэтому строка подключения минимальна:

```
Data Source=shop.db
```

Если файл не существует, SQLite создаст его автоматически при первом подключении. Можно указать абсолютный путь или оставить относительный — тогда файл создастся рядом с исполняемым файлом приложения. Для хранения базы только в оперативной памяти используется специальное значение — это удобно для тестов:

```
Data Source=:memory:
```

Строку подключения можно дополнить параметрами поведения. Для большинства приложений достаточно двух:

```
Data Source=shop.db;Foreign Keys=True
```

`Foreign Keys=True` — включает проверку внешних ключей, которая в SQLite по умолчанию отключена. Без этого параметра ограничения `FOREIGN KEY` в схеме будут описаны, но не будут проверяться при вставке и удалении данных.

В коде строка подключения передаётся в конструктор объекта соединения. Если путь к файлу задаётся динамически, удобнее использовать `SqliteConnectionStringBuilder` — это безопаснее, чем конкатенация вручную:

```
var builder = new SqliteConnectionStringBuilder
{
    DataSource  = "shop.db",
    ForeignKeys = true
};

string connectionString = builder.ToString();
// Data Source=shop.db;Foreign Keys=True
```

Где хранить строку подключения. Писать её прямо в коде не стоит: строка может содержать пароль, который попадёт в репозиторий. Правильный подход — выносить в конфигурацию.

Ключевые классы

ADO.NET строится вокруг нескольких базовых классов. Названия классов отличаются в зависимости от провайдера — для SQLite они начинаются с `Sqlite`, для PostgreSQL с `Npgsql`, для MS SQL Server с `Sql`, — но назначение и API у всех одинаковые.

`SqliteConnection` — управляет соединением с базой данных. Принимает строку подключения, открывает и закрывает физическое соединение.

`SqliteCommand` — представляет SQL-команду. Привязывается к соединению, принимает текст запроса и параметры.

`SqliteDataReader` — предназначен для чтения результатов запроса строка за строкой. Экономичен по памяти — данные не загружаются целиком.

`SqliteTransaction` — управляет транзакцией: позволяет зафиксировать (`Commit`) или отменить (`Rollback`) группу операций.

`SqliteDataAdapter` и `DataTable` — используются в отключённом режиме: загружают данные в память и позволяют работать с ними без активного соединения.

Все ключевые объекты ADO.NET реализуют интерфейс `IDisposable`: они владеют неуправляемыми ресурсами, которые необходимо явно освобождать. Правильный способ — конструкция `using`, которая гарантирует освобождение ресурса даже при исключении.

Два режима работы

ADO.NET поддерживает два принципиально разных способа работы с данными.

Подключённый режим (Connected Mode) — соединение остаётся открытым во время чтения данных. Данные читаются потоково через `SqliteDataReader`, строка за строкой, без загрузки всего результата в память. Это быстро и экономно, хорошо сочетается с асинхронным кодом и является основным режимом в современных приложениях.

```
SqliteConnection → SqliteCommand → SqliteDataReader
```

Отключённый режим (Disconnected Mode) — данные загружаются в память через `SqliteDataAdapter`, после чего соединение закрывается. Дальнейшая работа происходит с объектами `DataTable` в памяти без активного соединения.

```
SqliteConnection → SqliteDataAdapter → DataTable → (работа без соединения)
```

4. Подключённый режим

SqliteCommand — выполнение запросов

`SqliteCommand` представляет SQL-команду, которую нужно выполнить. Есть два равнозначных способа создания:

```
// Способ 1: через конструктор
using var command = new SqliteCommand("SELECT Id, Title FROM Products",
connection);

// Способ 2: через соединение
using var command = connection.CreateCommand();
command.CommandText = "SELECT Id, Title FROM Products";
```

В зависимости от типа запроса используется один из трёх асинхронных методов выполнения.

`ExecuteNonQueryAsync` — для команд, которые не возвращают данных: `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`. Возвращает количество затронутых строк:

```
using var command = connection.CreateCommand();
command.CommandText = "UPDATE Products SET Price = 49000 WHERE Id = 1";

int rowsAffected = await command.ExecuteNonQueryAsync(token);
Console.WriteLine($"Обновлено строк: {rowsAffected}");
```

`ExecuteScalarAsync` — для запросов, возвращающих одно значение: `COUNT`, `MAX`, `SUM` и т.д.:

```
using var command = connection.CreateCommand();
command.CommandText = "SELECT COUNT(*) FROM Products WHERE Price < 5000";

long count = (long)(await command.ExecuteScalarAsync(token))!;
Console.WriteLine($"Товаров дешевле 5000: {count}");
```

`ExecuteReaderAsync` — для запросов, возвращающих набор строк (`SELECT`). Возвращает `SqliteDataReader`:

```
using var command = connection.CreateCommand();
command.CommandText = "SELECT Id, Title, Price FROM Products";

using var reader = await command.ExecuteReaderAsync(token);
```

Параметризованные запросы

Никогда не подставляйте данные в запрос через конкатенацию строк. Это открывает уязвимость **SQL-инъекции** — злоумышленник может передать строку, которая изменит структуру запроса:

```
// Плохо — SQL-инъекция
string email = "' OR '1'='1";
string query = "SELECT * FROM Customers WHERE Email = '" + email + "'";
// Итоговый запрос вернёт все строки таблицы
```

Правильный подход — **параметризованные запросы**: SQL-код и данные передаются раздельно, база данных сама безопасно подставляет значения. Параметры обозначаются символом `@`:

```
using var command = connection.CreateCommand();
command.CommandText = "SELECT * FROM Customers WHERE Email = @email";
command.Parameters.Add("@email", SqliteType.Text).Value = email;
```

Доступны два способа добавления параметров. `Add` с явным указанием типа — предпочтительный вариант, он точнее контролирует тип передаваемого значения. `AddWithValue` — короче, но тип определяется автоматически, что иногда приводит к неожиданному поведению:

```
// Предпочтительно — явное указание типа
command.Parameters.Add("@price", SqliteType.Real).Value    = 55000.0;
command.Parameters.Add("@id",    SqliteType.Integer).Value = 1;
command.Parameters.Add("@name",  SqliteType.Text).Value    = "Иван";

// Короче, но тип определяется автоматически
command.Parameters.AddWithValue("@title", "Ноутбук");
```

Если значение может быть `null`, нужно передавать `DBNull.Value`:

```
command.Parameters.AddWithValue("@phone", phone ?? (object)DBNull.Value);
```

SqliteDataReader — чтение результатов

`SqliteDataReader` читает результаты запроса строка за строкой. Метод `ReadAsync()` перемещает указатель на следующую строку и возвращает `false`, когда строки закончились:

```
using var reader = await command.ExecuteReaderAsync();

while (await reader.ReadAsync())
{
    // читаем данные текущей строки
}
```

Данные из текущей строки можно читать тремя способами.

По индексу столбца — самый быстрый вариант:

```
int    id    = reader.GetInt32(0);
string title = reader.GetString(1);
double price = reader.GetDouble(2);
```

По имени столбца через `GetOrdinal` — получаем индекс по имени, затем читаем типизированно:

```
int    id    = reader.GetInt32(reader.GetOrdinal("Id"));
string title = reader.GetString(reader.GetOrdinal("Title"));
```

По имени столбца через индексатор — самый лаконичный способ, возвращает `object`, поэтому требует приведения типа:

```
int    id    = (int)(long)reader["Id"];    // SQLite возвращает INTEGER как
long
string title = (string)reader["Title"];
double price = (double)reader["Price"];
```

Обратите внимание: `reader["Id"]` возвращает `long`, а не `int` — это особенность SQLite, где тип `INTEGER` внутри всегда хранится как 64-битное число. Поэтому нужно двойное приведение `(int)(long)`, либо используйте `Convert.ToInt32(reader["Id"])` — он справляется с этим автоматически

Типизированные методы для основных типов SQLite:

```
reader.GetInt32(index);    // INTEGER → int
reader.GetInt64(index);    // INTEGER → long
```

```
reader.GetDouble(index);   // REAL    → double
reader.GetString(index);   // TEXT    → string
reader.GetBoolean(index);  // INTEGER (0/1) → bool
```

Если столбец может содержать `NULL`, перед чтением нужно проверить:

```
string? phone = reader.IsDBNull(3) ? null : reader.GetString(3);
```

Полный пример

Рассмотрим полный пример работы с таблицей `Products` — создание, чтение, обновление и удаление записей. Для хранения строки подключения и создания соединений используем вспомогательный класс:

```
public class Product
{
    public int    Id    { get; set; }
    public string Title { get; set; } = "";
    public double Price { get; set; }
    public int    Stock { get; set; }
}
```

```
public class Database(string connectionString)
{
    public SqliteConnection CreateConnection() =>
        new SqliteConnection(connectionString);
}
```

Инициализация схемы:

```
static async Task InitDatabaseAsync(Database db, CancellationToken token =
default)
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = """
        CREATE TABLE IF NOT EXISTS Products (
            Id    INTEGER PRIMARY KEY AUTOINCREMENT,
            Title TEXT    NOT NULL,
            Price REAL    NOT NULL CHECK (Price > 0),
            Stock INTEGER NOT NULL DEFAULT 0 CHECK (Stock >= 0)
        )
        """;
```

```
    await command.ExecuteNonQueryAsync(token);
}
```

Create — добавление записи с возвратом нового Id:

```
static async Task<int> InsertProductAsync(
    Database db,
    Product product,
    CancellationToken token = default)
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = """
        INSERT INTO Products (Title, Price, Stock)
        VALUES (@title, @price, @stock)
        RETURNING Id
        """;

    command.Parameters.Add("@title", SqliteType.Text).Value    =
product.Title;
    command.Parameters.Add("@price", SqliteType.Real).Value    =
product.Price;
    command.Parameters.Add("@stock", SqliteType.Integer).Value =
product.Stock;

    var result = await command.ExecuteScalarAsync(token);
    return Convert.ToInt32(result);
}
```

Read — чтение всех записей:

```
static async Task<List<Product>> GetAllProductsAsync(
    Database db,
    CancellationToken token = default)
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = "SELECT Id, Title, Price, Stock FROM Products
ORDER BY Title";

    using var reader = await command.ExecuteReaderAsync(token);

    var products = new List<Product>();
    while (await reader.ReadAsync(token))
```

```
    {
        products.Add(new Product
        {
            Id    = reader.GetInt32(0),
            Title = reader.GetString(1),
            Price = reader.GetDouble(2),
            Stock = reader.GetInt32(3)
        });
    }

    return products;
}
```

Read — чтение одной записи по Id:

```
static async Task<Product?> GetProductByIdAsync(
    Database db,
    int id,
    CancellationToken token = default)
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = "SELECT Id, Title, Price, Stock FROM Products
WHERE Id = @id";
    command.Parameters.Add("@id", SqliteType.Integer).Value = id;

    using var reader = await command.ExecuteReaderAsync(token);

    if (!await reader.ReadAsync(token))
        return null;

    return new Product
    {
        Id    = reader.GetInt32(0),
        Title = reader.GetString(1),
        Price = reader.GetDouble(2),
        Stock = reader.GetInt32(3)
    };
}
```

Update — обновление записи:

```
static async Task<bool> UpdateProductAsync(
    Database db,
    Product product,
    CancellationToken token = default)
```

```
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = """
        UPDATE Products
        SET Title = @title,
            Price = @price,
            Stock = @stock
        WHERE Id = @id
        """;

    command.Parameters.Add("@title", SqliteType.Text).Value    =
product.Title;
    command.Parameters.Add("@price", SqliteType.Real).Value    =
product.Price;
    command.Parameters.Add("@stock", SqliteType.Integer).Value =
product.Stock;
    command.Parameters.Add("@id",    SqliteType.Integer).Value = product.Id;

    int rowsAffected = await command.ExecuteNonQueryAsync(token);
    return rowsAffected > 0; // false если запись не найдена
}
```

Delete — удаление записи:

```
static async Task<bool> DeleteProductAsync(
    Database db,
    int id,
    CancellationToken token = default)
{
    using var connection = db.CreateConnection();
    await connection.OpenAsync(token);

    using var command = connection.CreateCommand();
    command.CommandText = "DELETE FROM Products WHERE Id = @id";
    command.Parameters.Add("@id", SqliteType.Integer).Value = id;

    int rowsAffected = await command.ExecuteNonQueryAsync(token);
    return rowsAffected > 0;
}
```

Использование:

```
var db = new Database("Data Source=shop.db;Foreign Keys=True");
await InitDatabaseAsync(db);
```

```
// Добавляем товары
int id1 = await InsertProductAsync(db, new Product { Title = "Ноутбук",
Price = 55000, Stock = 10 });
int id2 = await InsertProductAsync(db, new Product { Title = "Мышь",
Price =  1500, Stock = 50 });
int id3 = await InsertProductAsync(db, new Product { Title = "Клавиатура",
Price =  3000, Stock = 30 });

// Читаем все
var products = await GetAllProductsAsync(db);
foreach (var p in products)
    Console.WriteLine($"[{p.Id}] {p.Title} — {p.Price:C} (склад:
{p.Stock})");

// Обновляем
await UpdateProductAsync(db, new Product { Id = id1, Title = "Ноутбук Pro",
Price = 65000, Stock = 8 });

// Читаем одну запись
var product = await GetProductByIdAsync(db, id1);
Console.WriteLine($"Обновлён: {product?.Title} — {product?.Price:C}");

// Удаляем
await DeleteProductAsync(db, id3);
Console.WriteLine($"Товаров в базе: {(await
GetAllProductsAsync(db)).Count}");
```

5. Отключённый режим

В предыдущем разделе мы работали в подключённом режиме: соединение оставалось открытым на время чтения, данные читались строка за строкой через `SqliteDataReader`. Это эффективно для большинства сценариев, но иногда удобнее загрузить данные в память целиком, закрыть соединение и работать с ними локально. Именно для этого существует **отключённый режим**.

DataTable

`DataTable` — это представление одной таблицы в памяти. Он содержит коллекцию столбцов (`Columns`) и строк (`Rows`), поддерживает добавление, изменение и удаление записей, отслеживает состояние изменений каждой строки.

Создать `DataTable` можно вручную:

```
var table = new DataTable("Products");

table.Columns.Add("Id",    typeof(int));
table.Columns.Add("Title", typeof(string));
table.Columns.Add("Price", typeof(double));

// Добавление строки через массив значений
table.Rows.Add(1, "Ноутбук", 55000.0);

// Добавление строки через NewRow
DataRow row = table.NewRow();
row["Id"]    = 2;
row["Title"] = "Мышь";
row["Price"] = 1500.0;
table.Rows.Add(row);
```

Чтение данных из строк:

```
foreach (DataRow row in table.Rows)
{
    int    id    = (int)row["Id"];
    string title = (string)row["Title"];
    double price = (double)row["Price"];

    Console.WriteLine($"[{id}] {title} — {price:C}");
}
```

DataSet

`DataSet` — это контейнер для нескольких `DataTable`. По сути, `DataSet` — это мини-база данных в памяти. Между таблицами можно определять отношения через `DataRelation`:

```
var dataSet = new DataSet("Shop");
dataSet.Tables.Add(new DataTable("Products"));
dataSet.Tables.Add(new DataTable("Orders"));

// Связь Orders.ProductId → Products.Id
var relation = new DataRelation(
    "OrderProducts",
    dataSet.Tables["Products"]!.Columns["Id"]!,
    dataSet.Tables["Orders"]!.Columns["ProductId"]!
);
dataSet.Relations.Add(relation);
```

SqliteDataAdapter

`SqliteDataAdapter` — связующее звено между базой данных и `DataTable`. Он выполняет два действия: загружает данные из БД в память (`Fill`) и синхронизирует изменения обратно в базу (`Update`).

```
using var connection = new SqliteConnection(connectionString);
using var adapter    = new SqliteDataAdapter(
    "SELECT Id, Title, Price, Stock FROM Products",
    connection
);

var table = new DataTable();
adapter.Fill(table); // соединение открывается и закрывается автоматически
```

После `Fill` соединение закрыто — дальнейшая работа идёт только с `table` в памяти.

Загрузка данных и работа с ними

Загружаем таблицу и работаем с ней в памяти. Параметр `MissingSchemaAction.AddWithKey` указывает адаптеру подтянуть информацию о первичном ключе — это необходимо для корректного отслеживания изменений:

```
using var connection = new SqliteConnection(connectionString);
using var adapter    = new SqliteDataAdapter(
    "SELECT Id, Title, Price, Stock FROM Products",
    connection
```

```
);

adapter.MissingSchemaAction = MissingSchemaAction.AddWithKey;

var table = new DataTable("Products");
adapter.Fill(table);
```

После загрузки работаем с данными без соединения:

```
// Изменяем цену первого товара
table.Rows[0]["Price"] = 49000.0;

// Добавляем новую строку
DataRow newRow  = table.NewRow();
newRow["Title"] = "Монитор";
newRow["Price"] = 25000.0;
newRow["Stock"] = 5;
table.Rows.Add(newRow);

// Помечаем вторую строку для удаления
table.Rows[1].Delete();
```

Каждая строка `DataTable` хранит своё состояние — `DataRowState`:

```
foreach (DataRow row in table.Rows)
{
    if (row.RowState == DataRowState.Deleted) continue;

    Console.WriteLine($"{row["Title"]} — состояние: {row.RowState}");
    // Unchanged, Modified, Added, Deleted
}
```

Синхронизация изменений с базой

Чтобы сохранить изменения из `DataTable` обратно в базу, нужно настроить команды адаптера. Самый простой способ — использовать `SqliteCommandBuilder`, который генерирует команды `INSERT`, `UPDATE` и `DELETE` автоматически на основе `SELECT`-запроса:

```
using var connection = new SqliteConnection(connectionString);
using var adapter    = new SqliteDataAdapter(
    "SELECT Id, Title, Price, Stock FROM Products",
    connection
);

// CommandBuilder автоматически генерирует INSERT/UPDATE/DELETE
using var builder = new SqliteCommandBuilder(adapter);
```

```
// Update анализирует состояние каждой строки и выполняет нужные команды
int rowsAffected = adapter.Update(table);
Console.WriteLine($"Синхронизировано строк: {rowsAffected}");
```

`adapter.Update` перебирает строки `DataTable` и для каждой выполняет нужную команду в зависимости от `DataRowState`: `Added` → `INSERT`, `Modified` → `UPDATE`, `Deleted` → `DELETE`, `Unchanged` — пропускается.

Вручную управлять состоянием изменений можно через:

```
table.AcceptChanges(); // принять все изменения — состояние сбросится в
Unchanged
table.RejectChanges(); // отменить все несохранённые изменения
```

Фильтрация и поиск в DataTable

`DataTable` поддерживает фильтрацию и сортировку в памяти без обращения к базе:

```
// Select возвращает массив строк по условию
DataRow[] cheapProducts = table.Select("Price < 5000");
foreach (DataRow row in cheapProducts)
    Console.WriteLine(row["Title"]);

// С сортировкой
DataRow[] sorted = table.Select("Stock > 0", "Price DESC");

// Поиск по первичному ключу — быстрее, чем Select
DataRow? found = table.Rows.Find(1);
if (found != null)
    Console.WriteLine($"Найден: {found["Title"]}");
```

`DataView` — более гибкий инструмент для представления отфильтрованных данных:

```
var view = new DataView(table)
{
    RowFilter = "Price < 5000",
    Sort      = "Title ASC"
};

Console.WriteLine($"Дешёвых товаров: {view.Count}");
foreach (DataRowView rowView in view)
    Console.WriteLine(rowView["Title"]);
```

Полный пример

```
const string ConnectionString = "Data Source=shop.db;Foreign Keys=True";

// Загрузка
using var connection = new SqliteConnection(ConnectionString);
using var adapter    = new SqliteDataAdapter(
    "SELECT Id, Title, Price, Stock FROM Products",
    connection
);
adapter.MissingSchemaAction = MissingSchemaAction.AddWithKey;
using var builder = new SqliteCommandBuilder(adapter);

var table = new DataTable("Products");
adapter.Fill(table);
Console.WriteLine($"Загружено: {table.Rows.Count} товаров");

// Повышаем цену всех товаров на 10%
foreach (DataRow row in table.Rows)
    row["Price"] = (double)row["Price"] * 1.1;

// Добавляем новый товар
DataRow newProduct  = table.NewRow();
newProduct["Title"] = "Наушники";
newProduct["Price"] = 8000.0;
newProduct["Stock"] = 15;
table.Rows.Add(newProduct);

// Просматриваем изменения до сохранения
DataTable? changes = table.GetChanges();
if (changes != null)
{
    Console.WriteLine($"Изменений для сохранения: {changes.Rows.Count}");
    foreach (DataRow row in changes.Rows)
        Console.WriteLine($"  [{row.RowState}] {row["Title"]}");
}

// Сохраняем изменения в базу
int saved = adapter.Update(table);
Console.WriteLine($"Сохранено изменений: {saved}");

// Фильтрация в памяти
var view = new DataView(table)
{
    RowFilter = "Price > 10000",
    Sort      = "Price DESC"
};
```

```
Console.WriteLine($"Товаров дороже 10000: {view.Count}");
foreach (DataRowView row in view)
    Console.WriteLine($"  {row["Title"]} - {row["Price"]:C}");
```

Когда использовать отключённый режим

Оффлайн-редактирование — пользователь редактирует набор данных в интерфейсе, а изменения сохраняются одним пакетом по кнопке «Сохранить». Типичный сценарий для десктопных приложений.

Экспорт и отчёты — данные загружаются целиком, обрабатываются в памяти и выгружаются в файл. Постоянное соединение не нужно.

Кэширование — небольшие справочники (города, категории, статусы) загружаются один раз при старте и используются многократно без повторных запросов к базе.

6. Транзакции

При работе с базой данных часто требуется выполнить несколько операций как единое целое. Классический пример — перевод денег между счетами: нужно списать сумму с одного счёта и зачислить на другой. Если между этими двумя операциями произойдёт сбой, деньги исчезнут — спишутся, но не зачислятся. Именно для таких случаев существуют **транзакции**.

Транзакция — это группа операций с базой данных, которая выполняется как единое целое: либо все операции завершаются успешно, либо ни одна из них не применяется.

ACID

Транзакции обладают четырьмя фундаментальными свойствами, известными как **ACID**.

Atomicity (атомарность) — транзакция неделима. Все операции внутри неё выполняются полностью или не выполняются вовсе. Не бывает ситуации, когда часть операций применилась, а часть — нет.

Consistency (согласованность) — транзакция переводит базу данных из одного корректного состояния в другое. Все ограничения целостности соблюдаются до и после транзакции.

Isolation (изолированность) — параллельные транзакции не видят промежуточных изменений друг друга. Каждая транзакция работает так, как будто она единственная.

Durability (надёжность) — зафиксированные изменения сохраняются даже при сбое системы. После `Commit` данные гарантированно записаны.

BeginTransaction, Commit, Rollback

В ADO.NET транзакция создаётся через метод `BeginTransaction()` на открытом соединении. Каждая команда, которая должна выполняться в рамках транзакции, явно привязывается к ней через свойство `Transaction`.

```
using var connection = new SqliteConnection(connectionString);
await connection.OpenAsync(token);

using var transaction = connection.BeginTransaction();

try
{
    using var command = connection.CreateCommand();
    command.Transaction = transaction; // привязываем команду к транзакции

    // выполняем операции
```

```
    transaction.Commit(); // фиксируем все изменения
}
catch
{
    transaction.Rollback(); // отменяем все изменения при ошибке
    throw;
}
```

`Commit()` — фиксирует все изменения, сделанные в рамках транзакции. После этого они становятся постоянными.

`Rollback()` — отменяет все изменения транзакции, возвращая базу в состояние до её начала. Вызывается при любой ошибке в блоке `catch`.

Пример: перевод средств между счетами

Классический пример транзакции — банковский перевод. Все три операции должны выполниться вместе или не выполниться вовсе:

```
public async Task TransferAsync(
    int fromAccountId,
    int toAccountId,
    decimal amount,
    CancellationToken token = default)
{
    if (amount <= 0)
        throw new ArgumentException("Сумма перевода должна быть больше
нуля");

    using var connection = new SqliteConnection(_connectionString);
    await connection.OpenAsync(token);
    using var transaction = connection.BeginTransaction();
    try
    {
        // Шаг 1: списываем сумму с отправителя
        // Условие Balance >= @amount защищает от отрицательного баланса
        using var deductCmd = connection.CreateCommand();
        deductCmd.Transaction = transaction;
        deductCmd.CommandText = """
            UPDATE Accounts
            SET Balance = Balance - @amount
            WHERE Id = @id AND Balance >= @amount
            """;
        deductCmd.Parameters.AddWithValue("@amount", amount);
        deductCmd.Parameters.Add("@id", SqliteType.Integer).Value =
fromAccountId;

        int deducted = await deductCmd.ExecuteNonQueryAsync(token);
```

```
        if (deducted == 0)
            throw new InvalidOperationException("Недостаточно средств или
счёт не найден");

        // Шаг 2: зачисляем сумму получателю
        using var addCmd = connection.CreateCommand();
        addCmd.Transaction = transaction;
        addCmd.CommandText = """
            UPDATE Accounts
            SET Balance = Balance + @amount
            WHERE Id = @id
            """;
        addCmd.Parameters.AddWithValue("@amount", amount);
        addCmd.Parameters.Add("@id", SqliteType.Integer).Value =
toAccountId;

        int added = await addCmd.ExecuteNonQueryAsync(token);
        if (added == 0)
            throw new InvalidOperationException("Счёт получателя не
найден");

        // Шаг 3: записываем историю перевода
        using var logCmd = connection.CreateCommand();
        logCmd.Transaction = transaction;
        logCmd.CommandText = """
            INSERT INTO Transfers (FromAccountId, ToAccountId, Amount,
CreatedAt)
            VALUES (@from, @to, @amount, datetime('now'))
            """;
        logCmd.Parameters.Add("@from",   SqliteType.Integer).Value =
fromAccountId;
        logCmd.Parameters.Add("@to",     SqliteType.Integer).Value =
toAccountId;
        logCmd.Parameters.AddWithValue("@amount", amount);

        await logCmd.ExecuteNonQueryAsync(token);

        // Все три операции прошли успешно — фиксируем
        transaction.Commit();
        Console.WriteLine($"Перевод {amount:C} выполнен успешно");
    }
    catch
    {
        // Любая ошибка — откатываем всё
        transaction.Rollback();
        throw;
    }
}
```

Если на любом из трёх шагов возникнет ошибка, `Rollback` вернёт базу в исходное состояние: баланс отправителя останется прежним, баланс получателя не изменится, запись в истории не появится.

Пример: создание связанных записей

Транзакции нужны не только для финансовых операций, но и для взаимосвязанных данных. Создадим заказ с несколькими позициями и одновременно спишем товары со склада:

```
public async Task<int> CreateOrderAsync(
    int customerId,
    List<(int ProductId, int Qty)> items,
    CancellationToken token = default)
{
    using var connection = new SqliteConnection(_connectionString);
    await connection.OpenAsync(token);

    using var transaction = connection.BeginTransaction();

    try
    {
        // Шаг 1: создаём заказ
        using var orderCmd = connection.CreateCommand();
        orderCmd.Transaction = transaction;
        orderCmd.CommandText = """
            INSERT INTO Orders (CustomerId, Status)
            VALUES (@customerId, 'new')
            RETURNING Id
            """;
        orderCmd.Parameters.Add("@customerId", SqliteType.Integer).Value =
customerId;

        int orderId = Convert.ToInt32(await
orderCmd.ExecuteScalarAsync(token));

        // Шаг 2: добавляем позиции и списываем товары со склада
        foreach (var (productId, qty) in items)
        {
            // Получаем цену и проверяем наличие
            using var priceCmd = connection.CreateCommand();
            priceCmd.Transaction = transaction;
            priceCmd.CommandText = "SELECT Price, Stock FROM Products WHERE
Id = @id";
            priceCmd.Parameters.Add("@id", SqliteType.Integer).Value =
productId;

            using var reader = await priceCmd.ExecuteReaderAsync(token);
```

```
            if (!await reader.ReadAsync(token))
                throw new InvalidOperationException($"Товар #{productId} не
найден");

            double price = reader.GetDouble(0);
            int    stock = reader.GetInt32(1);

            if (stock < qty)
                throw new InvalidOperationException(
                    $"Недостаточно товара #{productId} на складе: " +
                    $"запрошено {qty}, доступно {stock}"
                );

            await reader.CloseAsync();

            // Добавляем позицию в заказ
            using var itemCmd = connection.CreateCommand();
            itemCmd.Transaction = transaction;
            itemCmd.CommandText = """
                INSERT INTO OrderItems (OrderId, ProductId, Qty, Price)
                VALUES (@orderId, @productId, @qty, @price)
                """;
            itemCmd.Parameters.Add("@orderId",   SqliteType.Integer).Value =
orderId;
            itemCmd.Parameters.Add("@productId", SqliteType.Integer).Value =
productId;
            itemCmd.Parameters.Add("@qty",       SqliteType.Integer).Value =
qty;
            itemCmd.Parameters.AddWithValue("@price", price);

            await itemCmd.ExecuteNonQueryAsync(token);

            // Списываем со склада
            using var stockCmd = connection.CreateCommand();
            stockCmd.Transaction = transaction;
            stockCmd.CommandText = """
                UPDATE Products
                SET Stock = Stock - @qty
                WHERE Id = @id
                """;
            stockCmd.Parameters.Add("@qty", SqliteType.Integer).Value = qty;
            stockCmd.Parameters.Add("@id",  SqliteType.Integer).Value =
productId;

            await stockCmd.ExecuteNonQueryAsync(token);
        }

        transaction.Commit();
        return orderId;
    }
```

```
    catch
    {
        transaction.Rollback();
        throw;
    }
}
```

Если какого-то товара не окажется на складе или возникнет любая другая ошибка — весь заказ откатится: ни заказ, ни позиции, ни изменения склада не сохранятся.

7. CRUD-операции и паттерн репозиторий

Паттерн Repository

В реальных приложениях код работы с базой данных не смешивают с бизнес-логикой. Для разделения используется паттерн **Repository** — класс, который инкапсулирует всю логику доступа к данным для одной сущности. Остальная часть приложения работает с репозиторием, не зная деталей SQL и ADO.NET.

Преимущества подхода: SQL сосредоточен в одном месте, его легко менять и тестировать, при смене базы данных достаточно поменять только репозиторий.

Структура проста: **интерфейс** описывает контракт — какие операции доступны, **класс реализации** содержит конкретный SQL-код.

Модель и схема

Возьмём простой пример — каталог книг.

```
public class Book
{
    public int    Id     { get; set; }
    public string Title  { get; set; } = "";
    public string Author { get; set; } = "";
    public int    Year   { get; set; }
    public double Price  { get; set; }
}
```

```
CREATE TABLE IF NOT EXISTS Books (
    Id     INTEGER PRIMARY KEY AUTOINCREMENT,
    Title  TEXT    NOT NULL,
    Author TEXT    NOT NULL,
    Year   INTEGER NOT NULL,
    Price  REAL    NOT NULL CHECK (Price >= 0)
);
```

Интерфейс репозитория

Интерфейс описывает контракт — стандартный набор CRUD-операций:

```
public interface IBookRepository
{
    Task<IEnumerable<Book>> GetAllAsync(CancellationToken token = default);
    Task<Book?> GetByIdAsync(int id, CancellationToken token = default);
    Task<int> CreateAsync(Book book, CancellationToken token = default);
    Task<bool> UpdateAsync(Book book, CancellationToken token = default);
    Task<bool> DeleteAsync(int id, CancellationToken token = default);
}
```

Реализация репозитория

Репозиторий принимает класс `Database` из раздела 4, который хранит строку подключения и создаёт соединения:

```
public class BookRepository : IBookRepository
{
    private readonly Database _db;

    public BookRepository(Database db)
    {
        _db = db;
    }
```

Инициализация схемы — создание таблицы при первом запуске. Удобно вынести в отдельный метод и вызвать один раз при старте приложения:

```
    public async Task InitializeAsync(CancellationToken token = default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = """
            CREATE TABLE IF NOT EXISTS Books (
                Id     INTEGER PRIMARY KEY AUTOINCREMENT,
                Title  TEXT    NOT NULL,
                Author TEXT    NOT NULL,
                Year   INTEGER NOT NULL,
                Price  REAL    NOT NULL CHECK (Price >= 0)
            )
            """;
```

```
        await command.ExecuteNonQueryAsync(token);
    }
```

GetAll — получение всех записей:

```
    public async Task<IEnumerable<Book>> GetAllAsync(CancellationToken token
= default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = "SELECT Id, Title, Author, Year, Price FROM
Books ORDER BY Title";

        using var reader = await command.ExecuteReaderAsync(token);

        var books = new List<Book>();
        while (await reader.ReadAsync(token))
            books.Add(MapBook(reader));

        return books;
    }
```

GetById — получение одной записи по Id:

```
    public async Task<Book?> GetByIdAsync(int id, CancellationToken token =
default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = """
            SELECT Id, Title, Author, Year, Price
            FROM Books
            WHERE Id = @id
            """;
        command.Parameters.Add("@id", SqliteType.Integer).Value = id;

        using var reader = await command.ExecuteReaderAsync(token);

        return await reader.ReadAsync(token) ? MapBook(reader) : null;
    }
```

Create — добавление записи с возвратом нового Id:

```
    public async Task<int> CreateAsync(Book book, CancellationToken token =
default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO Books (Title, Author, Year, Price)
            VALUES (@title, @author, @year, @price)
            RETURNING Id
            """;
        command.Parameters.Add("@title",  SqliteType.Text).Value    =
book.Title;
        command.Parameters.Add("@author", SqliteType.Text).Value    =
book.Author;
        command.Parameters.Add("@year",   SqliteType.Integer).Value =
book.Year;
        command.Parameters.Add("@price",  SqliteType.Real).Value    =
book.Price;

        var result = await command.ExecuteScalarAsync(token);
        return Convert.ToInt32(result);
    }
```

Update — обновление записи. Возвращает `false`, если запись с таким Id не найдена:

```
    public async Task<bool> UpdateAsync(Book book, CancellationToken token =
default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = """
            UPDATE Books
            SET Title  = @title,
                Author = @author,
                Year   = @year,
                Price  = @price
            WHERE Id = @id
            """;
        command.Parameters.Add("@title",  SqliteType.Text).Value    =
book.Title;
        command.Parameters.Add("@author", SqliteType.Text).Value    =
book.Author;
        command.Parameters.Add("@year",   SqliteType.Integer).Value =
```

```
book.Year;
        command.Parameters.Add("@price",  SqliteType.Real).Value    =
book.Price;
        command.Parameters.Add("@id",     SqliteType.Integer).Value =
book.Id;

        int rows = await command.ExecuteNonQueryAsync(token);
        return rows > 0;
    }
```

Delete — удаление записи по Id:

```
    public async Task<bool> DeleteAsync(int id, CancellationToken token =
default)
    {
        using var connection = _db.CreateConnection();
        await connection.OpenAsync(token);

        using var command = connection.CreateCommand();
        command.CommandText = "DELETE FROM Books WHERE Id = @id";
        command.Parameters.Add("@id", SqliteType.Integer).Value = id;

        int rows = await command.ExecuteNonQueryAsync(token);
        return rows > 0;
    }
```

Вспомогательный метод маппинга — преобразует текущую строку ридера в объект. Выносится в отдельный метод, чтобы не дублировать код в каждом методе чтения:

```
    private static Book MapBook(SqliteDataReader reader) => new()
    {
        Id     = reader.GetInt32(0),
        Title  = reader.GetString(1),
        Author = reader.GetString(2),
        Year   = reader.GetInt32(3),
        Price  = reader.GetDouble(4)
    };
}
```

Использование

```
var db   = new Database("Data Source=library.db;Foreign Keys=True");
var repo = new BookRepository(db);

await repo.InitializeAsync();

// Create
```

```
int id1 = await repo.CreateAsync(new Book { Title = "Мастер и Маргарита",
Author = "Булгаков",   Year = 1967, Price = 650 });
int id2 = await repo.CreateAsync(new Book { Title = "Преступление и
наказание", Author = "Достоевский", Year = 1866, Price = 490 });
int id3 = await repo.CreateAsync(new Book { Title = "Война и мир",
Author = "Толстой",    Year = 1869, Price = 890 });

// Read all
var books = await repo.GetAllAsync();
foreach (var b in books)
    Console.WriteLine($"[{b.Id}] {b.Title} — {b.Author}, {b.Year}
({b.Price:C})");

// Read one
var book = await repo.GetByIdAsync(id1);
Console.WriteLine($"Найдена: {book?.Title}");

// Update
bool updated = await repo.UpdateAsync(new Book
{
    Id     = id1,
    Title  = "Мастер и Маргарита",
    Author = "М. А. Булгаков",
    Year   = 1967,
    Price  = 720
});
Console.WriteLine($"Обновлено: {updated}");

// Delete
bool deleted = await repo.DeleteAsync(id3);
Console.WriteLine($"Удалено: {deleted}");
Console.WriteLine($"Книг в каталоге: {(await repo.GetAllAsync()).Count()}");
```

Вывод:

```
[2] Преступление и наказание — Достоевский, 1866 (490,00 ₽)
[1] Мастер и Маргарита — Булгаков, 1967 (650,00 ₽)
[3] Война и мир — Толстой, 1869 (890,00 ₽)
Найдена: Мастер и Маргарита
Обновлено: True
Удалено: True
Книг в каталоге: 2
```